Introduction to the Yale SOM HPC

A guide for researchers whose code has, until now, lived on a laptop

Contents

What is an HPC, and why use one? . . . 2
The mental model: a shared instrument . . . 2
Login nodes vs compute nodes . . . 3
Connecting to the cluster . . . 3
The job scheduler: thinking in jobs, not processes . . . 4
Right-sizing: the post-job feedback loop . . . 5
The filesystem zoo . . . 6
Moving files in and out of the HPC . . . 7
Software: modules, environments, and no sudo . . . 8
Running Python . . . 9
Running R . . . 10
Running Stata . . . 11
GPUs . . . 11
Working with large data . . . 12
Acquiring data: WRDS, APIs, scraping . . . 13
Starting a new project . . . 14
HPC etiquette . . . 14
Working with Claude Code on the HPC . . . 15
Getting help . . . 15
Appendix: example files . . . 17

> Claude Code skills: `overview`, `managing-jobs`
> This guide has a companion: the Yale SOM Claude Code marketplace, a plugin that ships fifteen skills — AI-facing counterparts to the sections below, covering `overview`, `managing-jobs`, `using-gpus`, `running-python`, `running-r`, `acquiring-data`, and the rest. When you ask Claude Code to do something on the cluster, it loads the matching skill and follows the same conventions this document teaches you. Each section ends with a callout naming the relevant skill or skills; Section 18 has the full map and install instructions.

What is an HPC, and why use one?

On a laptop, you have a fixed budget of cores, memory, and time, and you control all three. Once the job exceeds any one of them, you wait. The Yale SOM HPC exists to give you a much larger budget — and to give it to you on terms set by a shared scheduler rather than by your hardware.

An HPC ("high-performance computing") cluster is a collection of large servers — *compute nodes* — sitting in a data center, sharing a fast filesystem, and managed by a *job scheduler* that hands out CPU cores, memory, GPUs, and time to many users at once. The Yale SOM cluster lives at `hpc.som.yale.edu`. You log in to a small *login node*, and all real work runs on the compute nodes after the scheduler grants you a slot.

The cluster is the right tool when at least one of five things is true: your data does not fit in your laptop's RAM; a run takes long enough that you do not want to babysit your laptop through it; you want to run the same script hundreds of times with different parameters in parallel; you need a GPU you do not own; or the work is sensitive enough that it should live on Yale-managed storage rather than a personal device. The cluster is *not* the right tool for a 30-second script, for interactive plotting, or for work that fits comfortably on the machine you already have. The right move is to migrate to the cluster when the laptop stops being enough, not out of habit.

This guide does two things. First, it explains the mental model — what is different about a shared, scheduled, multi-tenant compute environment, and why almost every operational rule on the cluster follows from that one fact. Second, it provides the practical recipes — Slurm scripts, module loads, file-transfer patterns, language-specific workflows — you will use every day.

We structure the rest of the guide as follows. Sections 2–4 cover the mental model, the login-versus-compute distinction, and how to connect. Sections 5–7 cover Slurm, the post-job feedback loop, and the filesystem. Sections 8–9 cover moving files in and out and installing software. Sections 10–12 cover Python, R, and Stata. Sections 13–15 cover GPUs, larger-than-memory data, and acquiring data from outside the cluster. Sections 16–17 cover project setup and etiquette. Section 18 explains how this guide pairs with the Claude Code marketplace, which encodes the same conventions for the AI you delegate to. Section 19 lists the commands and contacts to keep at hand, and the appendix gives a complete first-job example.

> Claude Code skill: `overview`
> The `overview` skill is the AI's version of this section. When you ask Claude Code to do anything on the cluster without specifying further, it loads `overview` first to get its bearings.

The mental model: a shared instrument

The single biggest adjustment from working locally is that the HPC is shared, right now, with everyone else in SOM. Your laptop is yours. Latency is zero. Nothing is queued. You never wait. The HPC is the opposite of all of those, and operational rules on the cluster follow from this difference.

To see what that means in practice, four facts are worth keeping in mind together. CPUs, memory, and GPUs are a finite pool divided across many users, so an inflated request makes

you wait longer in the queue *and* blocks someone else. Resources are also reserved up front rather than discovered at runtime — you declare what you need before a job starts, and Slurm enforces the declaration by killing any job that exceeds it. Usage is public; `squeue` shows what every user on the cluster is running, so a 64-CPU job left idle overnight is visible by name to the people waiting behind it. And side effects propagate: millions of tiny files slow the GPFS metadata server for everyone, an aggressive scraper gets the cluster's outbound IP blocked for everyone, and a misbehaving login-node process lags everyone else's editor.

The Yale SOM HPC marketplace summarizes the implications as two pillars. *Be polite*: do not hog, do not sit idle, do not trash GPFS metadata, do not get the cluster IP-blocked. *Be skillful*: right-size requests, control threads, cache expensive work, make outputs resumable, and inspect resource use after every serious job. Everything that follows in this guide is an elaboration on one or the other.

Login nodes vs compute nodes

When you `ssh` to the cluster, you land on a *login node*. The login nodes are small machines shared by every logged-in user, and they exist for editing files, running `git`, browsing modules with `module`
`spider`, doing small package installs, and submitting jobs. They are not for running an analysis, training a model, loading a 5 GB CSV into pandas, or anything else that pegs a CPU or holds significant RAM. When someone runs heavy compute on a login node, every other user's editor lags and the sysadmins notice.

The right way to "just try something quickly" is an interactive job on a compute node:

```
srun --pty --cpus-per-task=2 --mem=4G --time=01:00:00 bash
```

This allocates two cores, 4 GB of memory, and one hour of walltime, and drops you into an interactive shell on a real compute node. Exit the moment you are done — `Ctrl+D` or `exit` — rather than letting it sit open while you go to a meeting.

> Claude Code skills: `overview`, `managing-jobs`
> Claude Code knows not to run heavy commands on the login node, and reaches for `srun --pty` or `sbatch` instead. If it ever proposes running a long-running command on a login node, push back.

Connecting to the cluster

The basic form of connecting is the same as any SSH session:

```
ssh <your-netid>@hpc.som.yale.edu
```

That works, but for daily use it is worth three small upgrades that turn SSH from "type my password every time" into something that disappears into the background.

First, generate an SSH key with a passphrase on your laptop using `ssh-keygen -t ed25519`, and copy the public half to the cluster with `ssh-copy-id <netid>@hpc.som.yale.edu`. Second, run an SSH agent — `ssh-agent` on Linux, the macOS keychain on a Mac — so that you type the passphrase once per session rather than once per `ssh` call. Third, add a host block to `~/.ssh/config` so that the full hostname is no longer required:

```
Host somhpc
  HostName hpc.som.yale.edu
  User <your-netid>
  ForwardAgent yes
```

After those three steps, `ssh somhpc`, `scp`, `rsync`, `git` over SSH, and VS Code Remote-SSH all work without further ceremony. Two additional patterns are worth mentioning but are deferred to the skill below: *port forwarding* for running a Jupyter notebook or a VS Code remote server on a compute node and using it from your laptop's browser, and *jump-host configuration* for reaching the cluster from off the Yale network without VPN.

> Claude Code skill: `connecting-securely`
> Full recipes for SSH keys, agent forwarding, jump hosts, and Jupyter/VS Code tunnels live in `connecting-securely`. Ask Claude Code to "set up an SSH config for the SOM HPC" and it will follow that skill.

The job scheduler: thinking in jobs, not processes

On your laptop you run a process — `python script.py` — and it starts immediately. On the HPC you submit a *job*: a description of what resources you need and what to run with them. Slurm queues the job, finds a compute node with a slot that fits, and only then starts your code. If your job tries to use more memory or time than you declared, Slurm kills it. The contract is simple: you predict what you need; Slurm enforces it.

The contract has four axes — CPU cores, memory, walltime, and GPUs — declared with the directives in Table 1.

Axis	*Slurm directive*
CPU cores	`--cpus-per-task=N`
Memory	`--mem=16G` (per node) or `--mem-per-cpu=4G`
Walltime	`--time=HH:MM:SS`
GPUs	`--gres=gpu:1` or `--gpus=1` (only when needed)

Table 1: The four resource axes of a Slurm job.

A fifth knob, `--ntasks`, is the one most likely to confuse a new user. The plain-English distinction is that `--ntasks=N` requests N independent processes, typically MPI ranks, while `--cpus-per-task=N` requests N cores assigned to a single process so it can run threads. SOM workflows are almost never MPI, so almost every sbatch script should start with `--ntasks=1 --cpus-per-task=N`. Reach for `--ntasks>1` only when you have multiple cooperating processes.

A *partition* is a queue tied to a particular set of nodes. The SOM cluster currently exposes five. The default, `default_queue`, mixes CPU-only and A40 GPU nodes and caps walltime at four hours, which makes it well-suited to short tests and most interactive work. `cpunormal` is CPU-only with longer walltimes and is the workhorse for batch CPU jobs. `gpunormal` carries RTX 8000 and A100 GPUs. The `h100` partition is a single node with four H100 GPUs and is the scarcest resource on the cluster — every H100-hour you take is compute someone else cannot use. The `build` partition exists for compiling software. The live view is `sinfo -s`, which is worth running before any production submission.

A minimal sbatch script that puts the contract on paper looks like:

```bash
#!/bin/bash
#SBATCH --job-name=test
#SBATCH --partition=default_queue
#SBATCH --time=00:10:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}

hostname
python --version
```

The thread-environment block matters more than it looks; we return to it in Section 10. Submit with `sbatch my_job.sh`, monitor your queue with `squeue --me`, inspect a particular job with `scontrol show job <id>`, and cancel with `scancel <id>`. Job states you will see in practice are `R` (running), `PD` (pending in the queue), and `CG` (completing).

If a job sits in `PD` for an unusually long time, ask why. The right diagnostic is `squeue --me --start`, which gives the estimated start time and the queueing reason, followed by `scontrol show job <id>` for the full record. The most common cause of pending forever is requesting resources that do not exist on any node: 256 GB of RAM when the largest node has 192 GB, or 64 cores on partitions whose nodes top out at 48. Slurm queues such a job indefinitely without warning that it is impossible. Match the request to real hardware by inspecting `sinfo -s` and `scontrol show node`.

After a job finishes, `sacct -j <id>` reports its history with whatever columns you ask for:

```bash
sacct -j <id> --format=JobID,JobName,State,Elapsed,MaxRSS,ReqMem,AllocCPUS
```

Two patterns worth knowing about up front cover most of the structured work on a cluster. *Job arrays* answer "I want to run this script 500 times with different parameters" — add `#SBATCH --array=1-500%50` and the `%50` caps concurrent tasks at 50, leaving slots for other users; inside the script, `$SLURM_ARRAY_TASK_ID` is the index. *Dependencies* answer "B should start when A finishes": `sbatch --dependency=afterok:<jobid-of-A> B.sh`.

> Claude Code skills: `managing-jobs`, `self-diagnosing-resource-use`
> The `managing-jobs` skill has the full pattern library — array throttling, dependency chains, partition selection, the right-sizing loop. Pair it with `self-diagnosing-resource-use` for post-mortems.

Right-sizing: the post-job feedback loop

Resource requests are guesses. The discipline that distinguishes a skillful HPC user from a wasteful one is checking the guess after the job finishes and adjusting next time. Two commands do most of the work:

```
seff <job-id>
sacct -j <job-id> --format=JobID,Elapsed,MaxRSS,ReqMem,AllocCPUS,State
```

What does "good" look like on the resulting numbers? CPU efficiency from `seff` should sit somewhere above 70%; if it reads 5%, you asked for cores you did not use, and the next submission should drop `--cpus-per-task`. Memory peak from `MaxRSS` should land around 50-70% of `ReqMem`; if you asked for 64 GB and used 4 GB, cut the request, and if you used 63 of 64, leave a bit more headroom. Walltime used should land around 50-80% of the `--time` you requested; wildly over-requesting walltime keeps you out of the short-job backfill slots the scheduler uses to fill gaps and lengthens your queue.

Why bother? Over-requesting is not free. It costs *you* a longer queue because Slurm has to find a bigger empty slot for your job, and it costs other users the resources sitting idle inside your allocation. Inspect every serious job; adjust the next submission. The habit is worth more in the aggregate than any single performance optimization.

> Claude Code skill: `self-diagnosing-resource-use`
> Ask Claude Code "did my last job use what it asked for?" and it will run `sacct` and `seff`, interpret the output, and propose a tighter request for next time.

The filesystem zoo

Your laptop has one disk. The cluster has several filesystems, with different rules and lifetimes, summarized in Table 2. Knowing which is which is the difference between a workflow that runs cleanly and one that quietly fills `$HOME`, breaks shared metadata, or loses intermediates when a node reboots.

Path	*What it is for*	*Lifetime*
`$HOME` (= `/gpfs/home/$USER`)	Code, configs, dotfiles, small things. Backed up.	Permanent
`/gpfs/project/<proj>/`	Shared team data, code, outputs. Request via `somit@yale.edu`.	Project lifetime
`/gpfs/scratch60/$USER/`	Large temporary working files; staging area.	Treat as ephemeral; clean it
`/tmp` on a compute node	Local fast scratch, just for this job.	Gone when the job ends

Table 2: The four filesystems you will use, and what each is for.

The implications are short. Code lives in `$HOME` or in `code/` under your project; data lives under `/gpfs/project/<proj>/data/` or `/gpfs/scratch60/$USER/`. `$HOME` is small and is the wrong place for 200 GB of intermediate files. Scratch is a staging area, not an archive — clean it when work is done.

The metadata-storm warning

GPFS is good at large files and bad at millions of tiny ones. Every file creation, listing, or stat call hits a metadata server shared by every user on the cluster. A workflow that writes one output file per row of a 2-million-row dataset will slow `ls` for every user, including the

sysadmins, until you stop. Three counter-patterns cover most of the situations where this comes up: write Parquet rather than per-row CSVs; tar or zip directories of small files when you are done with them; and for "many small intermediates inside a single job," write to the compute node's `/tmp` and copy a single tarball back to GPFS at the end.

Atomic writes and resumable outputs

Jobs can be killed. You exceed memory, the wallclock runs out, or the node reboots for maintenance. If your script writes results directly to `output.parquet` and is killed halfway, you now have a half-written `output.parquet` that looks like a finished file but is not. The safe pattern is to write to a temporary path on the same filesystem and then rename:

```
import os
tmp = "output.parquet.tmp"
df.write_parquet(tmp)
os.replace(tmp, "output.parquet")  # atomic on the same filesystem
```

Combine this with skip-if-exists at the top of each unit of work and the pipeline becomes resumable: re-running the same script after a kill or a crash picks up where it left off rather than restarting from zero.

> Claude Code skills: `using-the-filesystem`, `working-with-large-data`
> `using-the-filesystem` has the full pattern library; `working-with-large-data` covers the Parquet / DuckDB / Polars patterns that avoid both metadata storms and out-of-memory kills.

Moving files in and out of the HPC

A new user's first concrete obstacle is rarely Slurm. It is getting an existing codebase and dataset onto the cluster in the first place, and getting results back off. Three situations cover most of what comes up.

I already have a codebase locally

Use Git, not `scp`. Push your repo to GitHub or to Yale's GitHub Enterprise, and on the cluster:

```
cd /gpfs/project/<proj>/
git clone git@github.com:<you>/<repo>.git code
```

From then on, you edit anywhere — laptop, cluster shell, VS Code Remote-SSH — and `git pull` and `git push` keep the two in sync. Two free wins follow that are worth stating explicitly. The first is an off-cluster backup of your code, with no admin involved: the SOM HPC does not give you user-controlled backups of `$HOME` or `/gpfs/project/`, but if your code is on GitHub then a cluster outage is irrelevant to your code's safety. The second is version history for the inevitable "what did I change last week." Commit before each serious run, print the commit hash into your job log, and `git bisect` when a result stops reproducing.

I have a dataset on my laptop

For one-shot transfers, `scp` is fine:

```
scp local.csv somhpc:/gpfs/project/<proj>/data/
```

For anything large, slow, or interruption-prone — which means most real datasets — prefer `rsync`, which resumes:

```
rsync -avP --partial local_dir/ somhpc:/gpfs/project/<proj>/data/
```

Do not check data into Git. That is what `/gpfs/project/` is for. (Git LFS exists; it is rarely worth the trouble for SOM workflows.)

I have results on the cluster I want back

Same tools, reversed direction:

```
rsync -avP --partial somhpc:/gpfs/project/<proj>/output/ ./output/
```

VS Code Remote-SSH makes single-file fetches transparent — opening the file in the remote editor downloads it as needed. The pattern worth avoiding is pulling thousands of small output files individually; tar them on the cluster first, both to spare GPFS metadata (Section 7) and to make the transfer one fast operation rather than thousands of slow ones:

```
tar -czf results.tar.gz output/
```

then `scp` the one tarball.

I want to fetch from a public URL

`wget` or `curl` straight onto `/gpfs/project/`, ideally from a compute node so the login node is not tied up. The cluster's outbound IP is shared, so credentialed downloads, APIs, and rate-limiting deserve the more careful treatment we give them in Section 15.

What does not belong on the HPC

Final-output PDFs, slide decks, Overleaf projects, and manuscript drafts belong on your laptop, in Overleaf, or in Dropbox. The cluster is for compute and the data feeding it. Build the figure on the cluster, copy it to your laptop, drop it into the paper.

> Claude Code skills: `connecting-securely`, `using-the-filesystem`, `starting-a-new-project`
> Ask Claude Code to "clone my GitHub repo into project space and set up a uv environment" and it will combine these three skills.

Software: modules, environments, and no sudo

The biggest software difference from your laptop is that you do not have `sudo`. You cannot `apt install`, `brew install`, or `pip install --system` your way out of a problem. Software on the cluster comes in three layers, in roughly this order of preference: cluster-managed modules, user-space environments, and containers.

The first layer is *modules*, served by Lmod and built behind the scenes with Spack, which give you the system-level tools — Python, R, Stata, Git, CUDA, MATLAB. Find software with `module spider <name>`, which prints every available version and the exact `module load` line for each, and load with `module load <name>` (or `module load <name>/<version>` for a specific build). View what is currently loaded with `module list`, and clear everything with `module purge`. SOM's modules are built with Spack as a back end, but as a user you only ever type `module ...`; if a tutorial tells you to run `spack install`, that is the admin's job, not yours.

The second layer is *project-local environments* for the packages your analysis actually uses. Use one environment per project, and put it in the project directory rather than in `$HOME`: for Python, `uv` is the modern, fast tool of choice (Section 10), with plain `python -m`

`venv` still serviceable; for R, `renv` per project (Section 11). The discipline matters because two projects with conflicting NumPy versions can otherwise corrupt each other in subtle ways that are miserable to debug.

The third layer is *containers*, via Apptainer, for the cases where modules and user-space environments are not enough — typically a GLIBC-too-old error, a stack that demands a different distribution, or a published artifact you want to reproduce bit-for-bit. This is a last resort, not a first one: modules and user-space envs cover the vast majority of cases.

> Claude Code skills: `installing-software`, `running-python`, `running-r`
> `installing-software` is the umbrella skill; the language-specific ones layer on top. Ask Claude Code to "install package X" and it will pick the right layer (module, uv, renv, Apptainer) for you.

Running Python

Loading Python on the cluster is one line:

```
module load python
```

For a real project, create a dedicated environment under the project directory rather than installing into the system Python:

```
python -m venv /gpfs/project/<proj>/.venv
source /gpfs/project/<proj>/.venv/bin/activate
pip install numpy pandas pyarrow polars duckdb
```

A typical Python sbatch script looks like the following. Note the thread-environment block, which we explain immediately afterwards:

```
#!/bin/bash
#SBATCH --job-name=py_analysis
#SBATCH --partition=cpunormal
#SBATCH --time=02:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}

module load python
source /gpfs/project/<proj>/.venv/bin/activate

python analysis.py
```

Why the thread-environment block matters

NumPy, SciPy, scikit-learn, and most of the rest of the scientific Python stack call into BLAS under the hood. By default, BLAS spawns one thread per CPU it sees on the *node*, not per CPU you *requested* from Slurm. To see how this goes wrong, suppose you ask for `--cpus-per-task=4` on a 96-core node. A single `numpy.linalg.solve` call can launch 96 BLAS threads, blow past

your CPU allocation, hammer the other users' jobs sharing that node, and run *slower* than four threads because of contention. The four `*_NUM_THREADS` exports in the block above pin BLAS to exactly the cores Slurm gave you. The `${SLURM_CPUS_PER_TASK:-1}` form, with the `:-1` fallback, matters: a bare `$SLURM_CPUS_PER_TASK` is empty when you run the script outside Slurm, which silently sets the variable to empty and produces a different bug.

`uv` for fast, reproducible environments

For new projects, prefer `uv` over `pip + venv`. It is faster, it produces a lockfile that pins every transitive dependency, and it collapses the `python -m venv` and `pip install` dance into a single tool. The marketplace skill below has the full recipe.

Resumable outputs

The atomic-write pattern from Section 7 generalizes to all batch Python work. Write a function that skips its work when the output file already exists, writes intermediate output to `<name>.tmp`, and calls `os.replace` to rename to `<name>` only after the write succeeds. Re-running the script after a kill or a crash then becomes idempotent rather than restarting work from zero.

> Claude Code skills: `running-python`, `parallel-python`, `accelerating-python`
> Three layered skills. `running-python` covers the basic Slurm + uv
> 1. thread-control pattern. `accelerating-python` covers DuckDB,
>
> Polars, and Numba, which are usually the right move *before* reaching for parallelism. `parallel-python` covers multiprocessing / joblib / Dask sizing once you actually need workers.

Running R

R loads in much the same way as Python:

```
module load r
```

Project-local package management uses `renv`:

```
module load r
cd /gpfs/project/<proj>/
R -e 'install.packages("renv", repos="https://cloud.r-project.org")'
R -e 'renv::init()'
```

A typical R sbatch script:

```
#!/bin/bash
#SBATCH --job-name=r_analysis
#SBATCH --partition=cpunormal
#SBATCH --time=02:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
```

```
module load r
cd /gpfs/project/<proj>/
Rscript analysis.R
```

The BLAS-threads warning from Section 10 applies in R as well: `data.table`, matrix algebra, and any package that links against BLAS will silently oversubscribe a node without the `*_NUM_THREADS` exports. R also respects an explicit thread count inside the script itself — `data.table::setDTthreads(as.integer(Sys.getenv("SLURM_CPUS_PER_TASK")))` is the equivalent move at the language level.

> Claude Code skill: `running-r`
> `running-r` covers `renv` setup, batch invocation, and the BLAS, OpenMP, and `data.table` thread-control checklist.

Running Stata

Stata on the cluster runs in batch mode:

```
#!/bin/bash
#SBATCH --job-name=stata_run
#SBATCH --partition=cpunormal
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

module load stata
cd /gpfs/project/<proj>/

stata-mp -b do analysis.do
```

The `-b` flag puts Stata in batch mode, with output going to `<script>.log` rather than the terminal. Three Stata-specific notes matter. The number of MP cores Stata is configured to use should match the `--cpus-per-task` you asked Slurm for — asking Stata for 16 cores when Slurm gave you 4 just oversubscribes. Stata/MP cores come from a shared license pool, so do not grab 32 cores when 4 will do; the next person waiting for Stata is a colleague. And `tempfile` intermediates and large temporary files belong on scratch, not in `$HOME`.

> Claude Code skill: `running-stata`
> `running-stata` covers batch invocation, log handling, MP-core sizing, and the license-pool etiquette.

GPUs

Request a GPU only when your code actively uses CUDA — PyTorch, JAX, TensorFlow, RAPIDS, or a CUDA kernel you wrote yourself. A GPU sitting in a job that runs pure-NumPy code is the most expensive form of resource hoarding on the cluster, because GPUs are the scarcest resource and idle ones queue every other GPU user behind you.

A minimal GPU sbatch:

```
#!/bin/bash
#SBATCH --job-name=train
#SBATCH --partition=gpunormal
#SBATCH --gres=gpu:1
#SBATCH --time=04:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=32G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail
module load python cuda
source /gpfs/project/<proj>/.venv/bin/activate

nvidia-smi   # log GPU type at the top of the run
python train.py
```

The `h100` partition holds one node with four H100 GPUs — the scarcest resource on the cluster. Treat each H100-hour as compute someone else cannot use, and pick `gpunormal` (RTX 8000 / A100) unless you need the H100.

The cardinal sin: idle interactive GPUs

The single worst etiquette violation on the cluster is allocating a GPU interactively and then leaving it open while you go to lunch, to sleep, or to a meeting. The GPU sits idle, blocks colleagues, and `squeue` makes the offender visible by name. Cancel interactive GPU sessions the moment you stop typing — `exit` from inside, or `scancel <job-id>` from outside.

Verify the GPU is actually being used

Once you are inside a GPU allocation, `nvidia-smi` is the right diagnostic. If `GPU-Util` stays near 0% during your training loop, you are paying for a GPU you are not using. The usual culprits are that the framework is not configured to use CUDA, the model is too small to saturate the device, or the bottleneck is data loading on the CPU rather than computation on the GPU.

> Claude Code skill: `using-gpus`
> `using-gpus` walks through GPU partition selection, verifying that CUDA is in fact being used, and the idle-GPU detection patterns Claude Code uses to flag wasted allocations.

Working with large data

If your data fits comfortably in your job's RAM, a simple `pandas.read_csv` or `read.csv` is fine. Once it stops fitting, two rules cover most of what you need. First, use columnar formats rather than row formats — convert CSVs to Parquet once, and read Parquet from then on. Parquet is smaller, faster, and supports column pruning and predicate pushdown. Second, use a query engine rather than whole-file loading; DuckDB, Polars in lazy mode, and Arrow can stream over a 50 GB Parquet dataset inside a 16 GB job, which `pandas.read_csv("50gb.csv")` cannot.

A canonical pattern, using DuckDB to push down both column selection and a date filter:

```
import duckdb
con = duckdb.connect()
out = con.sql("""
  SELECT firm_id, year, AVG(ret) AS mean_ret
```

```
  FROM '/gpfs/project/<proj>/data/returns.parquet'
  WHERE year BETWEEN 1990 AND 2020
  GROUP BY firm_id, year
""").pl()  # to Polars; or .df() for pandas
```

DuckDB reads only the columns and rows it needs, in a streaming fashion, and never materializes the full file in memory. This is the right default for any analysis that touches a dataset larger than the job's RAM.

> Claude Code skills: `working-with-large-data`, `accelerating-python`
> `working-with-large-data` covers Parquet conversion, DuckDB and Polars patterns, and chunked pipelines. `accelerating-python` covers when to add Numba or parallelism *after* you have fixed the data layer.

Acquiring data: WRDS, APIs, scraping

When a job downloads data — from WRDS, a REST API, or a public website — three HPC-specific issues show up that do not exist on a laptop.

The first is that every job on the SOM cluster leaves the network from the same IP address. If your scraper hits a site too aggressively, the site blocks that IP, which blocks every other user too. Throttle requests, respect `robots.txt`, and assume the site's rate limit applies to the cluster as a whole rather than just to you.

The second is that the expensive part of "download then process" is almost always the download. Cache responses by a hash of the request — URL plus headers plus body — so that a re-run never re-fetches:

```
import hashlib, json, pathlib, urllib.request

CACHE = pathlib.Path("/gpfs/project/<proj>/cache")
CACHE.mkdir(exist_ok=True)

def fetch(url):
    key = hashlib.sha256(url.encode()).hexdigest()
    f = CACHE / f"{key}.json"
    if f.exists():
        return json.loads(f.read_text())
    data = json.loads(urllib.request.urlopen(url).read())
    f.write_text(json.dumps(data))
    return data
```

The third is credentials. Never put a password, API key, or WRDS credential in a script, notebook, or Git repo. Use `~/.pgpass` for WRDS or Postgres, `~/.netrc` for HTTP basic auth, environment variables, or a secrets file with `chmod 600`. Anything that lands in Git is, in practice, public forever.

> Claude Code skill: `acquiring-data`
> `acquiring-data` covers WRDS connection pooling, API caching, scraping etiquette, and the credentials pattern in detail.

Starting a new project

A reproducible project layout under `/gpfs/project/<proj>/` saves hours later. The skeleton we recommend is:

```
/gpfs/project/myproj/
├── code/                # git repo, pushed to GitHub
├── data/                # raw inputs, read-only by convention
├── output/              # results, regeneratable from code+data
├── logs/                # Slurm logs (%x_%j.out)
├── slurm/               # sbatch scripts
├── cache/               # request-hash caches
└── .venv/               # uv/venv environment, not in git
```

Three rules make the layout work in practice. Git tracks `code/` and nothing else, with a `.gitignore` that excludes `.venv/`, `data/`, `output/`, `logs/`, and `cache/`. Data is read-only by convention, and outputs are regeneratable — deleting `output/` and re-running should rebuild it. And one environment per project: do not share a `$HOME`-level Python or R environment across unrelated projects, because conflicting package versions are easy to introduce and hard to debug.

> Claude Code skill: `starting-a-new-project`
> Ask Claude Code to "set up a new project under `/gpfs/project/`" and it will create this layout, initialize Git and uv, and write a starter sbatch script.

HPC etiquette

The HPC is shared and `squeue` is public. If you are a poor citizen of the cluster, expect emails from colleagues waiting on the resources you are holding.

The most expensive form of poor citizenship is letting resources sit idle — an interactive session left open overnight, a batch job whose 24-hour walltime overshoots a 30-minute task, or an interactive GPU allocated and then abandoned for a meeting. Cancel idle sessions immediately with `scancel <job-id>`; an over-allocation that finishes early can be cancelled the same way without waiting for the wallclock to catch up.

Three other patterns matter enough to call out. Do not hog: requesting 64 CPUs and 256 GB when the job needs 4 and 16 keeps your job in the queue longer *and* blocks others. Do not trash GPFS metadata: workflows that write millions of tiny files slow `ls` for every user on the cluster, including the sysadmins, and the right counter-pattern is Parquet plus tarballs (Section 7). And do not get the cluster IP-blocked — every job leaves the network from the same outbound IP, so an aggressive scraper or API client breaks downloads for everyone.

A practical sub-rule for collaborative work: email `somit@yale.edu` for a shared `/gpfs/project/<name>/` folder rather than scattering files across personal directories. Shared folders carry proper permissions, reduce duplication, and survive when one team member's account is offboarded.

Working with Claude Code on the HPC

This guide has a companion: the Yale SOM HPC marketplace (`yale-som-hpc/claude-code-marketplace`), a Claude Code plugin that ships a set of skills — instructions written for the AI — mirroring the sections of this document. When you ask Claude Code to do something on the cluster, it loads the matching skill and follows the conventions encoded there.

The two artifacts are deliberately complementary. This document teaches you, the human, the cluster's mental model and the operational rules well enough to read Claude's plans critically and to operate without it. The marketplace teaches the AI you delegate to, so that when you say "submit this as a Slurm job," Claude already knows about thread exports, partition selection, atomic writes, the cardinal sin of idle GPUs, and the rest. Neither replaces the other: the human who does not understand the cluster cannot evaluate the AI's output, and the AI without the marketplace will produce plausible-looking instructions that violate cluster conventions.

The map from "I want to…" to which skill loads is given in Table 3.

"I want to…"	*Skill*
Get oriented on the cluster	`overview`
Set up SSH keys, Jupyter tunnels	`connecting-securely`
Submit, array, or chain Slurm jobs	`managing-jobs`
Check whether my last job was wasteful	`self-diagnosing-resource-use`
Choose where to store files on GPFS	`using-the-filesystem`
Install software / fix a GLIBC error	`installing-software`
Create a new project layout	`starting-a-new-project`
Run a Python job	`running-python`
Parallelize Python work	`parallel-python`
Speed up slow Python	`accelerating-python`
Run an R job	`running-r`
Run a Stata job	`running-stata`
Request a GPU	`using-gpus`
Process larger-than-memory data	`working-with-large-data`
Download data, call APIs, query WRDS	`acquiring-data`

Table 3: Map from research task to the skill Claude Code will load.

For current install instructions, see the marketplace `README.md`. The short version, from inside Claude Code:

```
/plugin marketplace add yale-som-hpc/claude-code-marketplace
/plugin install hpc@yale-som-hpc
```

Getting help

The commands and contacts you will reach for most often are short enough to keep in one place. For job state and history, use `squeue`

`--me` for what is running now, `squeue --me --start` and `scontrol show job <id>` for diagnosing why something is pending, and `seff <id>` together with `sacct -j <id>` for resource usage after a run. For partition and node detail, `sinfo -s` is the live view. For account, storage, and project-folder requests, email `somit@yale.edu`. Reference documentation lives at slurm.schedmd.com for Slurm and lmod.readthedocs.io for the module system, and the marketplace skills serve as a structured second source for the operational patterns this guide describes.

Appendix: example files

The files below form a complete first job — a Python script, a Slurm submission script, and the commands to submit and inspect the run. They are designed to be copied verbatim, run end-to-end, and then modified.

fibonacci.py

```python
#!/usr/bin/env python3
"""
Compute Fibonacci numbers. A trivial Python script for testing
HPC submission end-to-end.
"""

import sys
import time

def fibonacci(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

def fibonacci_sequence(n):
    return [fibonacci(i) for i in range(n)]

if __name__ == "__main__":
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 20
    print(f"Computing first {n} Fibonacci numbers...")
    t0 = time.time()
    seq = fibonacci_sequence(n)
    elapsed = time.time() - t0
    for i, fib in enumerate(seq):
        print(f"F({i}) = {fib}")
    print(f"\nDone in {elapsed:.4f} s. F({n}) = {fibonacci(n)}")
```

submit_fibonacci.sh

```bash
#!/bin/bash
#SBATCH --job-name=fibonacci
#SBATCH --partition=default_queue
#SBATCH --output=logs/fibonacci_%j.out
#SBATCH --error=logs/fibonacci_%j.err
#SBATCH --time=00:10:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}

echo "Job started at: $(date)"
```

```
echo "Running on node: $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "========================================="

module load python
python fibonacci.py 30

echo "========================================="
echo "Job finished at: $(date)"
```

Running it

```
mkdir -p logs
sbatch submit_fibonacci.sh
squeue --me
# when the job finishes:
seff <job-id>
cat logs/fibonacci_<job-id>.out
```

The `seff` call at the end is the habit worth building from day one.